

TOV-informed joint priors in `bilby.gw.prior`: (m, Λ) and $(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda})$

Derivation and implementation notes

October 6, 2026

Abstract

The equation of state (EOS) correlates a neutron star’s mass with its tidal deformability Λ . These notes document three priors in `bilby/gw/prior.py` that carry that correlation from a gridded KDE of TOV mass– Λ sequences into a bilby run. `TOVJointDist` and `TOVJointPrior` put it on a single (m, Λ) pair, one instance per star. `TOVMassLambdaTildeJointDist` and `TOVMassLambdaTildeJointPrior` push two such pairs through the exact change of variables to the usual BNS sampling parameters $(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda})$. `TOVConditionalLambdaPrior` puts only the conditional $p(\Lambda | m)$ on each star’s deformability and leaves the masses to whatever prior the run already uses. For each, we derive the density evaluated by `ln_prob` and the unit-cube transform used by `rescale`, show that the two agree, and describe how the classes plug into bilby’s prior machinery. Bugs found and fixed along the way are recorded where they occur.

Contents

1	Overview	2
2	The input grid	2
2.1	File format	2
2.2	Tables derived at construction	2
2.3	From $\log_{10} \Lambda$ to Λ	3
3	Single star: <code>TOVJointDist</code>	3
3.1	Density	3
3.2	Prior transform	4
3.3	Is <code>ln_prob</code> the density that <code>rescale</code> samples from?	4
3.4	Using it for a binary	5
4	Binary: <code>TOVMassLambdaTildeJointDist</code>	5
4.1	Construction	5
4.2	The model in component space	5
4.3	The map to sampling parameters	6
4.4	The Jacobian factorises	6
4.4.1	Mass block	6
4.4.2	Tidal block	7
4.5	The density	7
4.6	Support and bounds	7
4.7	Prior transform	8

5	Conditional deformability: TOVConditionalLambdaPrior	8
5.1	Model	8
5.2	The conditional density, exactly normalised	9
5.3	Prior transform and CDF	9
5.4	Which parameters it conditions on	9
5.5	Masses outside the grid	10
5.6	Implementation notes	10
6	Integration with bilby’s joint-prior machinery	10
6.1	How a sampler reaches the classes	10
6.2	Fixes to the interface	11
6.3	Bounds	12
7	Verification	12
7.1	<code>verify_tov_joint_priors.py</code>	12
7.2	Checks through bilby’s sampling interface	13
7.3	<code>TOVConditionalLambdaPrior</code>	13
8	Usage	14

1 Overview

Each `Dist` implements the three methods bilby expects of a joint distribution: `_ln_prob` (density), `_rescale` (unit-cube transform, used by nested samplers) and `_sample` (random draws). A sampler only ever reaches them through the per-parameter `Prior` objects and a `PriorDict`; Section 6 covers that layer.

2 The input grid

2.1 File format

The EOS information enters through a single file, `prob.data/tov_mass_lambda_joint_kde-grid.npz`, loaded in `TOVJointDist.__init__`. It holds a two-dimensional KDE of TOV sequences over a population of equations of state:

$$\log_pdf[i, j] = \ln p(m_i, \ell_j), \quad \ell \equiv \log_{10} \Lambda, \quad (1)$$

on grids `mass_edges` ($N = 200$ points, $m \in [1.000, 3.476] M_\odot$) and `log10_lambda_edges` ($M = 200$ points, $\ell \in [0.568, 4.738]$). The grid is normalised, $\int p(m, \ell) dm d\ell = 1$. It is fitted offline because refitting the KDE to the full population per job, let alone per likelihood call, is prohibitive. Empty regions of the KDE are floored at $\ln p = \ln 10^{-300} = -690.8$ rather than $-\infty$.

2.2 Tables derived at construction

`TOVJointDist.__init__` builds three objects from the grid, all by trapezoidal quadrature (`scipy.integrate.trapezoid` and `cumulative_trapezoid`).

The density interpolant. `self._log_pdf_interp` is a SciPy `RegularGridInterpolator` over (m, ℓ) , linear in $\ln p$ (so geometric in p), with `fill_value=-inf` off the grid. It is used only by `ln_prob`.

The marginal mass CDF.

$$p(m) = \int p(m, \ell) d\ell, \quad F(m) = \frac{\int_{m_{\min}}^m p(m') dm'}{\int_{m_{\min}}^{m_{\max}} p(m') dm'}, \quad (2)$$

tabulated at the mass grid points as `self.mass_cdf` and explicitly renormalised so that $F(m_{\max}) = 1$. It is exposed as `TOVJointDist.mass_cdf(mass)` (forward, linear interpolation) and `TOVJointDist.mass_from_cdf(u)` (inverse, linear interpolation of the same table with the axes swapped).

The conditional CDF of ℓ given m .

$$p(\ell | m_i) = \frac{p(m_i, \ell)}{p(m_i)}, \quad G_i(\ell) = \int_{\ell_{\min}}^{\ell} p(\ell' | m_i) d\ell', \quad (3)$$

one row per mass grid point, stored as `self.cond_cdf` with shape (N, M) and each row renormalised to end at 1. Rows whose marginal $p(m_i)$ vanishes are guarded against division by zero.

2.3 From $\log_{10} \Lambda$ to Λ

The grid is a density in $\ell = \log_{10} \Lambda$; every density these classes return is in Λ . With $d\ell/d\Lambda = 1/(\Lambda \ln 10)$,

$$p(m, \Lambda) = \frac{p(m, \ell)}{\Lambda \ln 10}, \quad \text{i.e.} \quad \ln p(m, \Lambda) = \ln p(m, \ell) - \ln \Lambda - \ln(\ln 10). \quad (4)$$

The last term is $\ln(\ln 10) = 0.8340$. Equation (4) is implemented once, in `TOVJointDist.ln_pdf_mass_lambda(mass, lam)`, which both `Dist` classes call.

Bug (fixed): $\ln 10$ in place of $\ln(\ln 10)$

The original `TOVJointDist.ln_prob` computed

$$\log_pdf_val - \text{np.log}(lam) - \text{np.log}(10.0)$$

that is, $\ln 10 = 2.3026$ where Eq. (4) requires $\ln(\ln 10) = 0.8340$. The log-density was low by the constant

$$\ln 10 - \ln(\ln 10) = \ln\left(\frac{10}{\ln 10}\right) = 1.4686$$

per (m, Λ) pair, a factor of 4.34 in density. Being constant, it did not bias posteriors, but it shifted log evidences and left the density unnormalised. It was found by the box estimator of Section 7: the four-parameter prior, which contains two pairs, came out at $18.8 \approx 4.34^2$ instead of 1. The fix moved the computation into `ln_pdf_mass_lambda` so that both classes share the corrected line.

3 Single star: TOVJointDist

3.1 Density

For parameters named `[mass_name, lambda_name]`, e.g. `["mass_1", "lambda_1"]`, the prior is the grid density itself, transformed to Λ :

$$\ln p(m, \Lambda) = \ln p(m, \ell)|_{\text{interp}} - \ln \Lambda - \ln(\ln 10), \quad (5)$$

evaluated by `TOVJointDist.ln_prob`, which forwards to `ln_pdf_mass_lambda` and then sets out-of-bounds samples to $-\infty$. The default `bounds` are the grid’s own extent, $m \in [m_{\min}, m_{\max}]$ and $\Lambda \in [10^{\ell_{\min}}, 10^{\ell_{\max}}]$, so the support is a box and coincides with where the interpolant is defined.

3.2 Prior transform

`TOVJointDist.rescale` maps $u = (u_0, u_1) \in [0, 1]^2$ to (m, Λ) by sequential inverse-CDF sampling, mass first and then ℓ given that mass:

$$m = F^{-1}(u_0), \quad \ell = G^{-1}(u_1 | m), \quad \Lambda = 10^\ell, \quad (6)$$

which samples $p(m)p(\ell | m) = p(m, \ell)$, as required. The first step is `mass_from_cdf`; the second is `log10_lambda_from_cdf(u, mass)`. Since G is tabulated only at the mass grid points, the latter blends the two neighbouring rows linearly in mass before inverting:

$$G(\ell | m) = (1 - w) G_i(\ell) + w G_{i+1}(\ell), \quad w = \frac{m - m_i}{m_{i+1} - m_i}, \quad m_i \leq m < m_{i+1}. \quad (7)$$

The blended row is still a monotone CDF, so the inversion is well defined. `_sample` draws u uniformly and calls `_rescale`, so random draws and the prior transform share one code path.

3.3 Is `ln_prob` the density that rescale samples from?

Both describe the same KDE, but they discretise it differently. `rescale` inverts piecewise-linear CDFs, so the density it actually samples is piecewise constant: on each mass cell,

$$p_r(m) = \frac{F(m_{i+1}) - F(m_i)}{m_{i+1} - m_i}, \quad p_r(\ell | m) = \frac{G(\ell_{j+1} | m) - G(\ell_j | m)}{\ell_{j+1} - \ell_j}, \quad (8)$$

with $G(\cdot | m)$ the blended row of Eq. (7). `ln_prob` instead interpolates $\ln p$ bilinearly. The two coincide at the grid points and differ inside cells, most where $\ln p$ changes quickly.

The size of the difference follows from importance-weighting draws from `rescale` by $w = p_{\text{lnprob}}/p_r$, both evaluated in (m, ℓ) . Then $\mathbb{E}[w] = \int p_{\text{lnprob}}$ is the normalisation of `exp(ln_prob)`, and the w -weighted quantiles are the quantiles a sampler driven by `ln_prob` (MCMC) converges to. With 2×10^6 draws:

	$\int \text{exp}(\text{ln_prob})$	largest quantile shift, 5–99%
<code>TOVJointDist</code> (m, Λ)	0.9987	0.3% (in Λ at 99%)
<code>TOVMassLambdaTildeJointDist</code>	0.9975	0.3% (in $\delta\tilde{\Lambda}$ at 99%)

For practical purposes the two are the same density. A nested sampler (driven by `rescale`) and an MCMC sampler (driven by `ln_prob`) target the same prior to well under a percent, and evidences computed with `exp(ln_prob)` are normalised to 0.25%.

Caveat: the density is not strictly positive on its own support

Linear inversion occasionally places a draw in a cell where the KDE was floored, so $\ln p \approx -690$ there. This affects about 10^{-5} of draws (6 in 4×10^5 for the four-parameter prior, with minimum $\ln p = -626$). It is harmless for sampling, but it makes estimators that divide by p , such as a smooth importance sampler $\mathbb{E}_p[q/p]$ with a Gaussian q , diverge. Weight-based diagnostics need a truncated proposal; the box estimator of Section 7 does not have this problem.

3.4 Using it for a binary

For a binary, create two independent instances, one per star:

```
d1 = TOVJointDist(grid, names=["mass_1", "lambda_1"])
d2 = TOVJointDist(grid, names=["mass_2", "lambda_2"])
for d in (d1, d2):
    for name in d.names:
        priors[name] = TOVJointPrior(d, name)
```

This models the two stars as independent draws from the EOS population, the same assumption as Eq. (9) below. It does *not* impose $m_1 \geq m_2$: half of all draws have `mass_2 > mass_1`. Runs that rely on bilby’s labelling convention should add a `Constraint` on `mass_ratio`, or use the four-parameter class, which imposes the ordering exactly.

The masses here come from the TOV distribution itself. To keep a run’s own mass prior and apply the EOS only to Λ , use `TOVConditionalLambdaPrior` (Section 5). Registering a `TOVJointPrior` for `lambda_1` alone, alongside an ordinary `mass_1` prior, does not do this. Because the joint prior waits for a mass it never receives, `rescale` fails with an inhomogeneous-shape error, `ln_prob` silently returns 0 for every Λ , and `sample()` draws Λ at a hidden mass of its own, uncorrelated with `mass_1`.

Bug (fixed): sibling instances merged during rescale

`ConditionalPriorDict.rescale` groups the parameters of a joint distribution by the string `dist.distname`, not by object identity. The base class sets `distname = "joint-dist"` for every instance, so the `mass_1/lambda_1` and `mass_2/lambda_2` pairs would be merged and their results mixed up. `TOVJointDist.__init__` now sets `distname = "tov-joint-" + "_" . join(names)`, which is unique per pair. `TOVMassLambdaTildeJointDist` does the same, with prefix `"tov_mc_q-lambda_tilde-"`.

4 Binary: TOVMassLambdaTildeJointDist

BNS runs are usually parametrised by $(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda})$, not by the component values. Giving $\tilde{\Lambda}$ and $\delta\tilde{\Lambda}$ independent uniform priors throws away the EOS correlation. This class builds the prior on $(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda})$ induced by the same grid. The construction is exact: both maps involved are invertible, so the density follows by change of variables, with no approximation beyond the grid itself.

4.1 Construction

`TOVMassLambdaTildeJointDist.__init__` takes the same `grid_file` and builds a private `TOVJointDist(grid_file, names=["mass", "lambda"])` as `self._grid`. Every grid operation below goes through that object’s methods of Section 2.2, so the two classes share one implementation of the tables and of Eq. (4). The internal names never reach the sampler. The default parameter names are `["chirp_mass", "mass_ratio", "lambda_tilde", "delta_lambda_tilde"]`, in that order; custom names must keep that order.

4.2 The model in component space

Write $\theta = (m_1, m_2, \Lambda_1, \Lambda_2)$. The two stars are independent draws from the same EOS population, conditioned on the labelling convention $m_1 \geq m_2$:

$$p(\theta) = 2p(m_1, \Lambda_1)p(m_2, \Lambda_2)\mathbb{I}[m_1 \geq m_2]. \quad (9)$$

The factor of 2 is the usual ordering factor: restricting an exchangeable pair to the half-space $m_1 \geq m_2$ doubles the density there.

This is a modelling choice worth stating plainly. It treats the two stars as independent samples from the *population* of EOS-drawn stars, not as two stars sharing one EOS. A common-EOS prior would correlate Λ_1 and Λ_2 beyond what their masses already imply; Eq. (9) does not.

4.3 The map to sampling parameters

Let $x = (\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda})$ with

$$\mathcal{M} = \frac{(m_1 m_2)^{3/5}}{(m_1 + m_2)^{1/5}}, \quad q = \frac{m_2}{m_1} \in (0, 1], \quad \eta = \frac{m_1 m_2}{(m_1 + m_2)^2} = \frac{q}{(1 + q)^2}. \quad (10)$$

The tidal combinations are those of Wade et al. (arXiv:1402.5156), as implemented in `bilby.gw.conversion.lambda_1_lambda_2_to_lambda_tilde` and `lambda_1_lambda_2_to_delta_lambda_tilde`:

$$\tilde{\Lambda} = \frac{8}{13} [c_1(\Lambda_1 + \Lambda_2) + c_2(\Lambda_1 - \Lambda_2)], \quad (11)$$

$$\delta\tilde{\Lambda} = \frac{1}{2} [c_3(\Lambda_1 + \Lambda_2) + c_4(\Lambda_1 - \Lambda_2)], \quad (12)$$

with η -dependent coefficients, returned by `lambda_tilde_coefficients(eta)`:

$$\begin{aligned} c_1 &= 1 + 7\eta - 31\eta^2, & c_2 &= \sqrt{1 - 4\eta} (1 + 9\eta - 11\eta^2), \\ c_3 &= \sqrt{1 - 4\eta} \left(1 - \frac{13272}{1319}\eta + \frac{8944}{1319}\eta^2\right), & c_4 &= 1 - \frac{15910}{1319}\eta + \frac{32850}{1319}\eta^2 + \frac{3380}{1319}\eta^3. \end{aligned} \quad (13)$$

The code clips $1 - 4\eta$ at zero before the square root, so round-off at $q = 1$ cannot produce a NaN.

4.4 The Jacobian factorises

The map $\theta \mapsto x$ has the block structure

$$\frac{\partial(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda})}{\partial(m_1, m_2, \Lambda_1, \Lambda_2)} = \begin{pmatrix} \frac{\partial(\mathcal{M}, q)}{\partial(m_1, m_2)} & \mathbf{0} \\ \frac{\partial(\tilde{\Lambda}, \delta\tilde{\Lambda})}{\partial(m_1, m_2)} & \frac{\partial(\tilde{\Lambda}, \delta\tilde{\Lambda})}{\partial(\Lambda_1, \Lambda_2)} \end{pmatrix}, \quad (14)$$

because the masses do not depend on the tidal parameters. The upper-right block vanishes, so the matrix is block triangular and its determinant is the product of the two diagonal blocks. The lower-left block, which is nonzero and depends on η , drops out entirely.

4.4.1 Mass block

With $m_1 = \mathcal{M}(1 + q)^{1/5} q^{-3/5}$ and $m_2 = qm_1$,

$$\frac{\partial m_1}{\partial \mathcal{M}} = \frac{m_1}{\mathcal{M}}, \quad \frac{\partial m_2}{\partial \mathcal{M}} = \frac{m_2}{\mathcal{M}} = \frac{qm_1}{\mathcal{M}}, \quad \frac{\partial m_2}{\partial q} = m_1 + q \frac{\partial m_1}{\partial q}, \quad (15)$$

so

$$\det \frac{\partial(m_1, m_2)}{\partial(\mathcal{M}, q)} = \frac{m_1}{\mathcal{M}} \left(m_1 + q \frac{\partial m_1}{\partial q} \right) - \frac{\partial m_1}{\partial q} \frac{qm_1}{\mathcal{M}} = \frac{m_1^2}{\mathcal{M}}. \quad (16)$$

The $\partial m_1 / \partial q$ terms cancel, leaving the same $m_1^2 / \mathcal{M}$ that appears in `bilby.gw.prior.convert_to_flat_in_component_mass_prior`.

4.4.2 Tidal block

At fixed masses the tidal map is *linear*:

$$\begin{pmatrix} \tilde{\Lambda} \\ \delta\tilde{\Lambda} \end{pmatrix} = A(\eta) \begin{pmatrix} \Lambda_1 \\ \Lambda_2 \end{pmatrix}, \quad A(\eta) = \begin{pmatrix} \frac{8}{13}(c_1 + c_2) & \frac{8}{13}(c_1 - c_2) \\ \frac{1}{2}(c_3 + c_4) & \frac{1}{2}(c_3 - c_4) \end{pmatrix}. \quad (17)$$

Hence

$$\begin{aligned} \det A &= \frac{8}{13} \cdot \frac{1}{2} \left[(c_1 + c_2)(c_3 - c_4) - (c_1 - c_2)(c_3 + c_4) \right] \\ &= \frac{4}{13} \left[-2c_1c_4 + 2c_2c_3 \right] = \frac{8}{13} (c_2c_3 - c_1c_4), \end{aligned} \quad (18)$$

a function of η alone, computed in log form by `_ln_abs_det_lambda_tilde_jacobian(eta)`. At equal masses $\eta \rightarrow 1/4$ and $\sqrt{1-4\eta} \rightarrow 0$, so $c_2, c_3 \rightarrow 0$ and $\det A \rightarrow -\frac{8}{13}c_1c_4 \neq 0$. The map stays invertible on the whole physical range, and no special case is needed at $q = 1$.

4.5 The density

Combining Eqs. (9), (16) and (18),

$$p(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda}) = 2p(m_1, \Lambda_1)p(m_2, \Lambda_2) \frac{m_1^2}{\mathcal{M}} \frac{1}{|\det A(\eta)|} \quad (19)$$

or, term by term as `TOVMassLambdaTildeJointDist._ln_prob` evaluates it,

$$\begin{aligned} \ln p(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda}) &= \ln 2 + \ln p(m_1, \Lambda_1) + \ln p(m_2, \Lambda_2) \\ &\quad + 2 \ln m_1 - \ln \mathcal{M} - \ln |\det A(\eta)|. \end{aligned} \quad (20)$$

The component values are recovered from x with the standard inversions, `chirp_mass_and-mass_ratio_to_total_mass`, `total_mass_and_mass_ratio_to_component_masses` and `lambda_tilde_delta_lambda_tilde_to_lambda_1_lambda_2` (all in `bilby.gw.conversion`). Each $\ln p(m_k, \Lambda_k)$ comes from `self._grid._ln_pdf_mass_lambda`, so it already includes the corrected Jacobian of Eq. (4). The bug of Section 2.3 entered this formula twice, once per star.

4.6 Support and bounds

The prior is *not* supported on a box in x . A rectangle in $(\tilde{\Lambda}, \delta\tilde{\Lambda})$ contains points whose implied Λ_1 or Λ_2 is negative, and points that fall outside the EOS grid entirely. `_ln_prob` builds a `valid` mask that requires $q \in (0, 1]$, $\mathcal{M} > 0$, $\Lambda_1, \Lambda_2 > 0$, and finite recovered components. At invalid points it substitutes harmless in-grid values before calling the interpolant, which keeps NaNs out of it, and then sets those points to $-\infty$. Points that are valid but off the grid get $-\infty$ from the interpolant's `fill.value`.

`TOVMassLambdaTildeJointDist.default_bounds()` returns *outer* bounds on this non-rectangular support; being inside them does not imply finite probability. They are derived as follows.

- $\mathcal{M}$ increases in both masses, so it is bracketed by $\mathcal{M}(m_{\min}, m_{\min})$ and $\mathcal{M}(m_{\max}, m_{\max})$.
- $q \in [m_{\min}/m_{\max}, 1]$.
- At fixed masses, $\tilde{\Lambda}$ and $\delta\tilde{\Lambda}$ are linear in (Λ_1, Λ_2) , so their extrema over the grid's Λ range lie at the four corners $\Lambda_k \in \{10^{\ell_{\min}}, 10^{\ell_{\max}}\}$. These are scanned over a 200×200 grid of ordered masses $m_1 \geq m_2$.

The tidal bounds come from a scan, not a proof. Empirically, none of 4×10^5 draws from `rescale` fell outside them.

Caveat: exact unit-cube edges

At $u_k \in \{0, 1\}$ exactly, `rescale` returns a point on the grid boundary. The round trip through the inversions above can then land $\sim 10^{-16}$ outside it, e.g. $m_1 = m_{\min} - 1.1 \times 10^{-16}$, and `ln_prob` returns $-\infty$. Continuous uniform draws essentially never hit these values; the interior of the cube is unaffected.

4.7 Prior transform

`TOVMassLambdaTildeJointDist.component_rescale` maps $u \in [0, 1]^4$ to θ , and `_rescale` then applies $\theta \mapsto x$ with `component_masses_to_chirp_mass`, $q = m_2/m_1$ and the two tidal conversions.

The ordering $m_1 \geq m_2$ is imposed by the transform itself, not by rejection, which keeps the map a smooth bijection. If m_1 is the larger of two independent draws from $p(m)$, its CDF is $F(m)^2$, so

$$m_1 = F^{-1}(\sqrt{u_0}), \quad m_2 = F^{-1}(u_1 F(m_1)), \quad (21)$$

the second being $p(m)$ truncated to $[m_{\min}, m_1]$. In code, these are `_grid.mass_from_cdf(np.sqrt(u0))` and `_grid.mass_from_cdf(u1 * _grid.mass_cdf(m1))`. The induced density is

$$\underbrace{2p(m_1)F(m_1)}_{\text{max of two draws}} \times \underbrace{\frac{p(m_2)}{F(m_1)}}_{\text{truncated}} = 2p(m_1)p(m_2), \quad (22)$$

exactly the mass part of Eq. (9); the $F(m_1)$ factors cancel. They also cancel for the discretised F the code actually uses, so the ordering adds no discretisation error beyond that of Section 3.3. The deformabilities then follow star by star from Eq. (6),

$$\Lambda_1 = 10^{G^{-1}(u_2|m_1)}, \quad \Lambda_2 = 10^{G^{-1}(u_3|m_2)}, \quad (23)$$

via `_grid.log10_lambda_from_cdf`.

Which cube coordinate feeds which physical parameter is arbitrary. `rescale` maps the whole vector at once, so the sampler never relies on a per-coordinate correspondence. As with the single-star class, `_sample` is `_rescale` applied to uniform draws.

5 Conditional deformability: TOVConditionalLambdaPrior

Both joint classes also fix the mass prior, to the EOS population's mass distribution. A run often has its own mass prior, e.g. uniform in component masses, and wants the EOS only to shape Λ given that mass. `TOVConditionalLambdaPrior` does this.

5.1 Model

With any mass prior $\pi(m_1, m_2)$ chosen by the user, the prior is

$$p(m_1, m_2, \Lambda_1, \Lambda_2) = \pi(m_1, m_2) p(\Lambda_1 | m_1) p(\Lambda_2 | m_2), \quad p(\Lambda | m) = \frac{p_{\text{TOV}}(m, \Lambda)}{p_{\text{TOV}}(m)}. \quad (24)$$

One instance is created per star (`component=1` or `2`, default name `lambda_<component>`). Each depends only on its own star's mass, so, as in Eq. (9), the two deformabilities are independent given the masses. Because the masses are conditioned on rather than transformed, no mass Jacobian enters. The prior on the masses is exactly π , and Λ is correlated with the mass the run actually samples.

5.2 The conditional density, exactly normalised

The joint density $p_{\text{TOV}}(m, \Lambda)$ is the one every other class uses, `TOVJointDist.ln_pdf_mass_lambda` (Eq. 5). Its normaliser $p_{\text{TOV}}(m) = \int p_{\text{TOV}}(m, \Lambda) d\Lambda$ is computed from the same interpolant, not from the trapezoid table of Eq. (2), so that each conditional integrates to exactly one.

The interpolant is bilinear in $\ln p$. At fixed mass m in the cell $[m_i, m_{i+1}]$, it is therefore piecewise *linear* in ℓ , with node values blended linearly in mass:

$$\ln p_{\text{TOV}}(m, \ell) = a_j + d_j t, \quad a_j = (1-w) \log_pdf[i, j] + w \log_pdf[i+1, j], \quad d_j = a_{j+1} - a_j, \quad (25)$$

for $\ell = \ell_j + t h_j$ with $t \in [0, 1]$, cell width $h_j = \ell_{j+1} - \ell_j$, and w as in Eq. (7). The density is piecewise exponential, so each cell integrates in closed form:

$$I_j = \int_{\ell_j}^{\ell_{j+1}} p_{\text{TOV}}(m, \ell) d\ell = h_j \frac{e^{a_{j+1}} - e^{a_j}}{d_j} \quad (\rightarrow h_j e^{a_j} \text{ as } d_j \rightarrow 0), \quad p_{\text{TOV}}(m) = \sum_j I_j. \quad (26)$$

Then $\int p_{\text{TOV}}(m, \Lambda) d\Lambda = \int p_{\text{TOV}}(m, \ell) d\ell$, because the $\ell \rightarrow \Lambda$ Jacobian of Eq. (4) cancels. `ln_prob` returns

$$\ln p(\Lambda | m) = \ln p_{\text{TOV}}(m, \Lambda) - \ln \sum_j I_j(m). \quad (27)$$

Numerically, each row is shifted by its maximum before exponentiating. The grid's log-density spans $[-690.8, 0.33]$, with jumps of up to 685 between neighbouring cells, so the unshifted form would overflow or underflow. `TOVConditionalLambdaPrior._conditional_tables` computes the shifted a_j , the cumulative sums $C_j = \sum_{k < j} I_k$, and $\ln p_{\text{TOV}}(m)$, vectorised over masses.

5.3 Prior transform and CDF

The same closed form gives the CDF and its inverse, with no separate tables. For ℓ in cell j at fraction t ,

$$G(\ell | m) = \frac{C_j + h_j (e^{a_j + d_j t} - e^{a_j}) / d_j}{\sum_k I_k}, \quad (28)$$

implemented by `TOVConditionalLambdaPrior.cdf`. For `rescale`, the target $r = u \sum_k I_k$ selects the cell with $C_j \leq r < C_{j+1}$. The remainder $\rho = r - C_j$ then gives

$$t = \frac{\ln(e^{a_j} + \rho d_j / h_j) - a_j}{d_j} \quad (\rightarrow \rho / (h_j e^{a_j}) \text{ as } d_j \rightarrow 0), \quad \Lambda = 10^{\ell_j + t h_j}. \quad (29)$$

The argument of the logarithm is evaluated directly, without dividing by e^{a_j} , so cells at the KDE floor are handled without overflow. Cells of zero width in probability are never selected.

Consequence. Unlike the joint classes (Section 3.3), `rescale` and `ln_prob` here describe *the same* density, not two discretisations of it. The verification in Section 7.3 confirms this to rounding error.

5.4 Which parameters it conditions on

`mass_parameters` selects the required variables, i.e. the keys `ConditionalPriorDict` must supply:

- "component_masses" (default): `mass_<component>`, for runs that sample the component masses directly.

- `"chirp_mass_and_mass_ratio"`: `chirp_mass` and `mass_ratio`, converted internally with `chirp_mass_and_mass_ratio_to_total_mass` and `total_mass_and_mass_ratio_to_component_masses`. This mode is for runs such as `prior_files/aligned_spins_bns.prior`, which sample $(\mathcal{M}, q)$ and hold `mass_1`, `mass_2` as `Constraints`. A constraint is not a sampled parameter, so it cannot be conditioned on, and `ConditionalPriorDict` would report the conditions as unresolvable. As the conditioning is on a deterministic function of $(\mathcal{M}, q)$, no Jacobian is needed here either.

For a given mass, both modes return the same density.

5.5 Masses outside the grid

$p(\Lambda | m)$ is defined only for $m \in [1.000, 3.476] M_\odot$. Outside that range, `ln_prob` returns $-\infty$, and `rescale` returns `nan` with a one-off warning. It does not raise, because bilby rescales every key before checking constraints, and an out-of-grid draw that a `Constraint` is about to reject must not abort the run. The mass prior should therefore lie inside the grid. In `"chirp_mass_and_mass_ratio"` mode, the component masses implied by a $(\mathcal{M}, q)$ box generally reach beyond it. The `mass_1` and `mass_2` constraints must then be tightened to the grid, e.g. to `[1.0, 3.47]`, instead of the `[0.5, 5]` in `aligned_spins_bns.prior`.

5.6 Implementation notes

- The class follows the idiom of `ConditionalChiInPlane` in the same file. It subclasses `ConditionalBasePrior`, sets `_required_variables` explicitly, passes a no-op `_condition_function`, and overrides `rescale`, `ln_prob`, `prob` and `cdf`. `sample` is inherited and calls `rescale` on uniform draws. Calls with the wrong required variables raise `IllegalRequiredVariablesException`. A call with none, i.e. outside a `ConditionalPriorDict`, raises a `ValueError` saying so.
- `conditional_prior_factory` renames the class on instantiation. The constructor restores the name, and `__repr__` and `get_instantiation_dict` use the `Prior` versions, so the prior round-trips through prior files and result JSON. `grid_file` is stored as given; use an absolute path if results will be reloaded elsewhere.
- `grid_file` may also be an existing `TOVJointDist`, whose loaded grid is then shared. `TOVJointDist.__init__` keeps the raw table as `_log_pdf` for this class.
- Scalar inputs return NumPy scalars, not 0-d arrays, for the LAL REAL8 reason given in Section 6.

6 Integration with bilby's joint-prior machinery

6.1 How a sampler reaches the classes

Each parameter has its own `Prior` object (`TOVJointPrior` or `TOVMassLambdaTildeJointPrior`), all sharing one `Dist`. Their constructors only add a type check on `dist` to `JointPrior.__init__`. The shared `Dist` holds per-call state, and the inherited `JointPrior` methods use it as follows.

- `rescale(val)` stores `val` in `dist.rescale_parameters` and returns `[]` until every parameter of the `dist` has been supplied. It then stacks the stored values *in `dist.names` order*, calls `dist.rescale`, and resets the store. The order in which a sampler lists its search keys therefore does not matter. `ConditionalPriorDict.rescale` then copies the full vector back to each parameter, grouping by `distname` (Section 3.4).

- `ln_prob(val)` works the same way through `dist.requested_parameters`. Every parameter but the last returns 0, and the last returns the joint log-density, so the sum `PriorDict.ln_prob` forms is correct.
- `sample()` draws all parameters of the `dist` at once on the first call and hands out the stored values on later calls, so the parameters of one draw belong to the same (m, Λ) or θ .

Nested samplers (`dynesty` and others) use only `rescale`; MCMC samplers (`emcee`, `bilby_mcmc`) use only `ln_prob`. Both feed the `log_prior` column of the result, which `Result` computes as `priors.ln_prob(dict=posterior), axis=0`, i.e. vectorised over samples.

The conditional prior takes a different route. `TOVConditionalLambdaPrior` holds no shared state and goes through `ConditionalPriorDict`'s conditional path. The dict recognises it by its `condition_func` attribute and orders it after the keys listed in `required_variables`. It then passes their most recently sampled values (`least_recently_sampled`) as keyword arguments to `rescale`, `ln_prob` and `sample`. For vectorised `ln_prob`, these are arrays, and the class broadcasts them against the Λ values.

6.2 Fixes to the interface

Four interface bugs made the original classes fail inside real runs. All are fixed in the subclasses, leaving `bilby/core/prior/joint.py` untouched so that the fork stays a clean addition on top of upstream. Each fix is harmless if upstream changes later.

Bug (fixed): `xp` keyword

Upstream `BaseJointPriorDist.ln_prob` now passes an array-namespace argument `xp` to `_ln_prob`. Both `_ln_prob` implementations are decorated with `@xp_wrap` and accept `xp`. They compute in NumPy, because `RegularGridInterpolator` is NumPy-only, and convert back with `xp.asarray`. The classes are therefore NumPy-bound: an `xp` of CuPy or JAX fails in `_ln_prob`.

Bug (fixed): `random_state` ignored

`_sample` drew from NumPy's global generator, so seeded runs did not reproduce and pool workers could share a stream. Both `_sample` methods now resolve the generator with `bilby.core.utils.random.resolve_random_state(random_state)`.

Bug (fixed): 0-d arrays from `sample()`

`JointPrior.sample` ends in `sample.squeeze()`, which for a size-1 draw returns a 0-d ndarray, not a float. LAL's SWIG REAL8 typemap rejects it:

```
TypeError: in method
'SimInspiralWaveformParamsInsertTidalLambda1',
argument 2 of type 'REAL8'
```

The masses escaped only because they are multiplied by `lal.MSUN_SI` before reaching LAL, which turns them into scalars. `TOVJointPrior.sample` and `TOVMassLambdaTildeJointPrior.sample` append `[]`, which unwraps 0-d arrays and leaves larger results unchanged.

Bug (fixed): shape-(1,) arrays from ln_prob

`BaseJointPriorDist.ln_prob` returns shape (1,) for a single sample, while ordinary priors return floats. `PriorDict.ln_prob` then fails while stacking the per-parameter terms:

```
ValueError: setting an array element with a sequence. ...
```

`TOVJointDist.ln_prob` and `TOVMassLambdaTildeJointDist.ln_prob` override it to return a scalar in that case, matching what `BaseJointPriorDist.rescale` already does.

6.3 Bounds

Caveat: tightened bounds are honoured by ln_prob only

Setting a narrower range on one parameter, e.g. `priors["lambda_tilde"].maximum = 600`, updates `dist.bounds`. `BaseJointPriorDist._check_samp` then sets `ln_prob` to $-\infty$ outside the range, but `BaseJointPriorDist.rescale` applies no bounds. In that example, 26% of `rescale` draws still have $\tilde{\Lambda} > 600$, so a nested sampler and an MCMC sampler would explore different priors. The truncated density is also no longer normalised.

There is a second, upstream problem in `_check_samp`: for a batch of samples, it keeps the out-of-bounds mask of the *first* parameter that has any violation and does not combine masks across parameters. With bounds tightened on two parameters, the vectorised `ln_prob` used for `log_prior` let 349 of 532 violating points through with finite probability. Evaluated one sample at a time, all of them correctly get $-\infty$.

Neither problem arises with the default bounds, where the support check in `_ln_prob` does the real work. To restrict the range, leave the bounds alone and add a `bilby.core.prior.Constraint` instead. Constraints are applied consistently and renormalised by `PriorDict`.

7 Verification

7.1 verify_tov_joint_priors.py

The script at the repository root runs four groups of checks on the `Dist` classes directly:

```
python verify_tov_joint_priors.py [path/to/tov_mass_lambda_joint_kde_grid.npz]
```

1. Jacobians against finite differences. At four configurations, including near-equal masses where $\sqrt{1-4\eta} \rightarrow 0$, the full 4×4 determinant of $\theta \mapsto x$ is compared with the analytic $m_1^2/\mathcal{M} \cdot |\det A|^{-1}$. They agree to better than 10^{-6} in the log, which also confirms the block-triangularity argument.

2. Ordering and invertibility. For 2×10^4 draws from `_component_rescale`, $m_1 \geq m_2$ and $q \in (0, 1]$ always hold. Pushing the draws back through the inversions of `_ln_prob` recovers θ to a relative error of 1.5×10^{-11} , and `ln_prob` is finite on all of them.

3. ln_prob is the density rescale samples from. This is the check that caught the bug of Section 2.3. For samples $x_i \sim p$ and any box B ,

$$\frac{1}{N} \sum_{x_i \in B} \frac{1}{\text{vol}(B) p(x_i)} \xrightarrow{N \rightarrow \infty} \int_B \frac{p(x)}{\text{vol}(B) p(x)} dx = 1. \quad (30)$$

This unbiased estimator of 1 does not depend on how sharply p varies inside B . A raw histogram comparison does, and for this strongly correlated density it is off by more than an order of magnitude at feasible sample sizes. Any missing constant, Jacobian or normalisation shows up directly as a departure from 1.

4. Control. The same estimator applied to the two-parameter `TOVJointDist`.

The box estimator only probes the bulk of the distribution. The global comparison of Section 3.3, done by importance-weighting draws with the exact density of Eq. (8), extends the agreement to the whole support, tails included.

7.2 Checks through bilby’s sampling interface

The script calls the `Dist` classes directly. The following was also checked through `BNSPriorDict` and `bilby.run_sampler`, exactly as a run calls the classes.

- `ConditionalPriorDict.rescale` reproduces `dist.rescale` for any ordering of the search keys, with fixed and ordinary priors mixed in, and leaves no state behind. `PriorDict.ln_prob` returns a scalar for single samples and the correct array for the vectorised `log_prior` computation. Single draws are scalars, and the four values of one draw are coherent.
- **dynesty**, four-parameter prior with a Gaussian likelihood on $(\mathcal{M}, \tilde{\Lambda})$: $\ln Z = -1.579 \pm 0.043$ against a Monte Carlo value of -1.562 . All 5/50/95% posterior quantiles of the four parameters match the importance-reweighted prior, and the `log_prior` column equals `dist.ln_prob`. The same holds with `npool=2` ($\ln Z = -1.583 \pm 0.059$), which confirms the classes pickle and work in spawned workers.
- **dynesty**, two `TOVJointDist` pairs: $\ln Z = -1.743 \pm 0.063$ against -1.734 , with quantiles in agreement.
- **emcee** with a flat likelihood reproduces the prior’s $\mathcal{M}$ and q to 2% of the 90% width. It shows tails in $\tilde{\Lambda}$ and $\delta\tilde{\Lambda}$ that are 2–8% heavy, which is slow mixing of the stretch move in long, correlated tails: Section 3.3 bounds any genuine `ln_prob/rescale` difference at 0.3%. MCMC runs dominated by the prior need long, thinned chains.

7.3 TOVConditionalLambdaPrior

Exactness.

- $\int \exp(\ln_prob) d\Lambda = 1$ to 4.5×10^{-8} at eight masses, including both grid edges. The residual comes from the fine numerical quadrature used to check it.
- $|\text{cdf}(\text{rescale}(u)) - u| < 1.3 \times 10^{-12}$ over 2×10^5 random (u, m) .
- `cdf` agrees with a direct numerical integral of $\exp(\ln_prob)$ to 3×10^{-8} . Together with the previous item, this shows `rescale` samples exactly the density `ln_prob` returns. KS tests of `rescale` draws against `cdf` at three masses give $p = 0.43$ – 0.81 .
- $\ln p_{\text{joint}}(m, \Lambda) - \ln p(\Lambda \mid m)$, using `TOVJointDist.ln_prob` for the joint, is constant in Λ to 4×10^{-14} . The conditional is therefore the joint renormalised, as Eq. (24) states.

Through BNSPriorDict.

- The Λ keys resolve as conditional.
- `rescale` and `ln_prob` match direct calls for any key order.

- `ln_prob` is a scalar for single samples and correct when vectorised.
- Single draws are NumPy scalars.
- `mass_1` keeps its own `Uniform` prior (KS), while $\text{corr}(m_1, \ln \Lambda_1) = -0.89$, so Λ follows the run's mass.
- The `"chirp_mass_and_mass_ratio"` mode gives the same density as the component mode at the implied mass.
- Off-grid masses give `nan` with a warning (`rescale`) and $-\infty$ (`ln_prob`).
- Wrong or missing required variables raise.
- The prior survives repr, a prior-file round trip and a JSON round trip.

Samplers. With uniform component masses on $[1, 2]$ and a Gaussian likelihood on (Λ_1, Λ_2) :

mass parameters	dynesty $\ln Z$	Monte Carlo $\ln Z$
<code>component_masses</code>	-1.014 ± 0.034	-1.003
<code>chirp_mass_and_mass_ratio</code>	-1.166 ± 0.035	-1.172

In both modes, all 5/50/95% posterior quantiles of the masses and deformabilities match the importance-reweighted prior. The `log_prior` column is finite, and the result reloads from JSON with the prior intact. `emcee` with a flat likelihood, which uses only `ln_prob`, reproduces the prior. With 64 walkers and 3×10^4 steps, the 5–95% quantiles deviate by 0.4% of the 90% width for m_1 and 2.4% for Λ_1 . A $5 \times$ shorter chain gave 5.1% for Λ_1 , with the upper tail on the other side of the reference. The residual is mixing in the heavy Λ tail, not a density error, which the exactness checks above rule out.

The fork's `requirements.txt` requires `dynesty>=3.0.0`, because `bilby.core.sampler.dynesty_utils` imports `dynesty.internal.samplers`; `dynesty 2.x` fails at import.

8 Usage

Four-parameter prior, for runs in $(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda})$:

```
from bilby.gw.prior import (
    BNSPriorDict, TOVMassLambdaTildeJointDist, TOVMassLambdaTildeJointPrior)

grid = "prob_data/tov_mass_lambda_joint_kde_grid.npz"
dist = TOVMassLambdaTildeJointDist(grid)
priors = BNSPriorDict() # then add the remaining BNS parameters
for name in dist.names:
    priors[name] = TOVMassLambdaTildeJointPrior(dist, name)
```

Per-star joint prior, for runs in component masses and deformabilities: see Section 3.4, and note that it does not order the masses.

Conditional Λ with your own mass prior, sampling component masses:

```
from bilby.core.prior import Uniform
from bilby.gw.prior import BNSPriorDict, TOVConditionalLambdaPrior

grid = "/abs/path/prob_data/tov_mass_lambda_joint_kde_grid.npz"
```

```
priors = BNSPriorDict(dictionary=dict(
    mass_1=Uniform(1.0, 2.0, "mass_1"),
    mass_2=Uniform(1.0, 2.0, "mass_2"),
    lambda_1=TOVConditionalLambdaPrior(grid, component=1),
    lambda_2=TOVConditionalLambdaPrior(grid, component=2),
)) # then add the remaining BNS parameters
```

or, in a prior file that samples $(\mathcal{M}, q)$, as `aligned_spins_bns.prior` does:

```
mass_1 = Constraint(name='mass_1', minimum=1.0, maximum=3.47)
mass_2 = Constraint(name='mass_2', minimum=1.0, maximum=3.47)
lambda_1 = bilby.gw.prior.TOVConditionalLambdaPrior(grid_file='/path/grid.npz',
    component=1, mass_parameters='chirp_mass_and_mass_ratio')
lambda_2 = bilby.gw.prior.TOVConditionalLambdaPrior(grid_file='/path/grid.npz',
    component=2, mass_parameters='chirp_mass_and_mass_ratio')
```

In all cases:

- for the joint classes, use one `Dist` instance per joint group and pass the same object to each of its `Priors`;
- restrict ranges with a `Constraint`, not by changing `minimum/maximum` (Section 6);
- keep every mass the EOS is evaluated at inside the grid, $[1.000, 3.476] M_{\odot}$;
- the classes are NumPy-only.

Class / function	Role
<code>TOVJointDist(grid_file, names, bounds=None)</code>	Joint distribution of one star's (m, Λ) ; owns the grid and all the tables derived from it (Sections 2–3).
<code>TOVJointPrior(dist, name)</code>	Per-parameter <code>JointPrior</code> wrapper; two per star.
<code>TOVMassLambdaTildeJointDist(grid_file, names=None, bounds=None)</code>	Joint distribution of $(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda})$ for a binary; holds a <code>TOVJointDist</code> internally as <code>_grid</code> and reuses its tables (Section 4).
<code>TOVMassLambdaTildeJointPrior(dist, name)</code>	Per-parameter <code>JointPrior</code> wrapper; four per binary.
<code>TOVConditionalLambdaPrior(grid_file, component, ...)</code>	Conditional prior $p(\Lambda m)$ on one star's deformability, for use with an independent mass prior; one per star (Section 5).
<code>_lambda_tilde_coefficients(eta)</code>	The coefficients $c_1, \dots, c_4$ of the tidal map (Eq. 13).
<code>_ln_abs_det_lambda_tilde_jacobian(eta)</code>	$\ln \det A(\eta) $ of the tidal map (Eq. 18).

Table 1: The public surface. Both `Dist` classes derive from `bilby.core.prior.BaseJointPriorDist`, and both `joint Prior` classes derive from `bilby.core.prior.JointPrior`. `TOVConditionalLambdaPrior` derives from `bilby.core.prior.ConditionalBasePrior` instead, because it conditions on parameters it does not own.

Box centre	before fix	current code
TOVMassLambdaTildeJointDist, $(\mathcal{M}, q, \tilde{\Lambda}, \delta\tilde{\Lambda})$		
(1.20, 0.85, 400, -30)	17.9 ± 1.2	1.02 ± 0.07
(1.35, 0.75, 250, -40)	19.0 ± 3.1	0.96 ± 0.08
(1.15, 0.90, 600, -20)	18.8 ± 1.2	1.06 ± 0.07
control: TOVJointDist, (m, Λ)		
(1.30, 400)	—	1.02 ± 0.02
(1.80, 100)	—	1.00 ± 0.01

Table 2: Box estimator, target 1, with $N = 4 \times 10^5$ samples. The “before fix” column was recorded when the bug was found. The constant 18.8 is $(10/\ln 10)^2$, the same error once per star.